

Optimización Evolutiva Multiobjetivo

Javier Ramírez González¹

CICESE, Ensenada, B.C., México
javier.ramirez@cicese.edu.mx

1 Ordenes y conos

Problema 1 (10 puntos). Sea $\|\cdot\| : \mathbb{R}^m \rightarrow \mathbb{R}_0^+$ una norma. Defina $x \leq_{\|\cdot\|} y \Leftrightarrow \|x\| \leq \|y\|$. ¿Es $\leq_{\|\cdot\|}$ un orden parcial? ¿Es conectado? Determine $C_{\leq_{\|\cdot\|}}$ para alguna norma $\|\cdot\|$ de su elección.

1.1 Preorden y conexidad

Definición. Dado un espacio vectorial con norma $\|\cdot\|$, definimos la relación $\leq_{\|\cdot\|}$ como:

$$x \leq_{\|\cdot\|} y \iff \|x\| \leq \|y\|.$$

Queremos determinar qué tipo de orden es esta relación.

Un orden parcial, debe cumplir:

- Reflexividad: $x \leq_{\|\cdot\|} x$ para todo x .
- Transitividad: si $x \leq_{\|\cdot\|} y$ e $y \leq_{\|\cdot\|} z$, entonces $x \leq_{\|\cdot\|} z$.
- Antisimetría: si $x \leq_{\|\cdot\|} y$ e $y \leq_{\|\cdot\|} x$, entonces $x = y$.

Reflexividad. Para cualquier x , se cumple $\|x\| \leq \|x\|$ (por la igualdad en $\mathbb{R}$). Por lo tanto, $x \leq_{\|\cdot\|} x$ siempre es verdadero. La relación es reflexiva.

Transitividad. Supongamos $x \leq_{\|\cdot\|} y$ e $y \leq_{\|\cdot\|} z$. Entonces

$$\|x\| \leq \|y\| \quad \text{y} \quad \|y\| \leq \|z\|.$$

La desigualdad $\leq$ en los números reales es transitiva, luego $\|x\| \leq \|z\|$. Esto equivale a $x \leq_{\|\cdot\|} z$. Así, la relación es transitiva.

Antisimetría Para que sea antisimétrica, de $\|x\| \leq \|y\|$ y $\|y\| \leq \|x\|$ debería deducirse $x = y$. Sin embargo, de esas dos desigualdades solo obtenemos $\|x\| = \|y\|$. En dimensión $m \geq 2$, existen vectores diferentes con la misma norma.

Ejemplo en $\mathbb{R}^2$ con la norma euclidiana:

Sean $x = (1, 0)^T$ e $y = (0, 1)^T$, se tiene:

$$\|x\|_2 = 1, \quad \|y\|_2 = 1.$$

Entonces $\|x\| \leq \|y\|$ y $\|y\| \leq \|x\|$ se cumplen (ambas son $1 \leq 1$). No obstante, $x \neq y$. Por lo tanto, la antisimetría se cumple.

La relación $\leq_{\|\cdot\|}$ es reflexiva y transitiva, pero no es antisimétrica. Por lo tanto, no es un orden parcial, sino un preorden.

Conexidad.

Una relación es conexa si para cualquier par x, y se cumple al menos una de las dos: $x \leq_{\|\cdot\|} y$ o $y \leq_{\|\cdot\|} x$.

Dados x e y , sus normas $\|x\|$ y $\|y\|$ son números reales no negativos. La relación $\leq$ en $\mathbb{R}$ es total: siempre es cierto que $\|x\| \leq \|y\|$ o $\|y\| \leq \|x\|$ (o ambas). Esto significa que, para cualquier par, se tiene $x \leq_{\|\cdot\|} y$ o $y \leq_{\|\cdot\|} x$. Por lo tanto, $\leq_{\|\cdot\|}$ es conexa.

La relación $\leq_{\|\cdot\|}$ es un **preorden total** (orden débil): es reflexiva, transitiva y total, pero no antisimétrica.

1.2 Cono local de direcciones de mejora

Definición. Sea $y \in \mathbb{R}^m$ con $y \neq 0$. Una dirección $d \in \mathbb{R}^m$ es de *mejora local* en y si existe un número $\bar{t} > 0$ tal que

$$\|y + td\|_2 \leq \|y\|_2 \quad \text{para todo } t \in (0, \bar{t}].$$

Es decir, al avanzar desde y una distancia pequeña a lo largo de d , la norma no aumenta.

Como la norma es no negativa, la desigualdad es equivalente a

$$\|y + td\|_2^2 \leq \|y\|_2^2.$$

Desarrollamos el cuadrado:

$$\langle y + td, y + td \rangle \leq \langle y, y \rangle.$$

Por propiedad del producto interno:

$$\langle y, y \rangle + 2t\langle y, d \rangle + t^2\|d\|_2^2 \leq \langle y, y \rangle.$$

Se simplifica $\langle y, y \rangle$ en ambos lados y obtenemos

$$2\langle y, d \rangle + t\|d\|_2^2 \leq 0. \tag{1}$$

Esta desigualdad debe cumplirse para todo t suficientemente pequeño.

Supongamos que $\langle y, d \rangle > 0$. Entonces el término $2\langle y, d \rangle$ es positivo. Para un t muy pequeño, $t\|d\|_2^2$ es despreciable, por lo que el lado izquierdo de (1) es positivo y la desigualdad falla. Por lo tanto, para que exista algún $\bar{t} > 0$, es necesario que

$$\langle y, d \rangle \leq 0.$$

– **Caso 1.** $\langle y, d \rangle < 0$.

Elegimos $\bar{t} = -\frac{2\langle y, d \rangle}{\|d\|_2^2}$ (> 0). Para cualquier $t \in (0, \bar{t}]$ se verifica que

$$2\langle y, d \rangle + t\|d\|_2^2 \leq 2\langle y, d \rangle + \bar{t}\|d\|_2^2 = 0,$$

por lo que la desigualdad (1) se cumple. Así, toda dirección con producto interno negativo es de mejora local.

– **Caso 2.** $\langle y, d \rangle = 0$.

Sustituyendo en (1) obtenemos $t\|d\|_2^2 \leq 0$. Como $t > 0$, esto solo es posible si $\|d\|_2^2 = 0$, es decir, $d = 0$. Por tanto, la única dirección con producto interno nulo que funciona es la dirección nula.

Para $y \neq 0$

El conjunto de direcciones de mejora local es:

$$\mathcal{C}_{\leq \|\cdot\|_2}(y) = \{d \in \mathbb{R}^m \mid \langle y, d \rangle < 0\} \cup \{0\}.$$

Son todos los vectores d que forman un ángulo mayor a 90° con y (apuntan en dirección opuesta a y), junto con el vector cero.

Para $y = 0$

Si $y = 0$, la condición $\|td\|_2 \leq 0$ para $t > 0$ pequeño solo se cumple cuando $d = 0$. Por lo tanto,

$$\mathcal{C}_{\leq \|\cdot\|_2}(0) = \{0\}.$$

2 ASF y cono de direcciones de mejora

Problema 2 (15 puntos). Sea la función achievement scalarizing function (ASF) definida como

$$s(x; w) = \max_{i=1, \dots, m} \left\{ \frac{x_i}{w_i} \right\},$$

donde $w \in \mathbb{R}^m$ es un vector de pesos convexo. Defina la relación binaria $x \leq_s y \Leftrightarrow s(x; w) \leq s(y; w)$. Construya el cono de direcciones de mejora $C_{\leq_s}$, describiéndolo explícitamente. Dibuje el cono asociado para el caso de dos dimensiones. Además, demuestre lo siguiente.

- (a) $C_{\leq_s}$ es convexo.
- (b) $C_{\leq_s}$ es puntiagudo.
- (c) Demuestre el tipo de orden inducido por la relación $\leq_s$.

Se define $x \leq_s y \Leftrightarrow s(x; w) \leq s(y; w)$. El cono global de direcciones de mejora $C_{\leq_s}$ se construye respecto al origen. Como $s(0; w) = 0$, una dirección d pertenece al cono si $s(d; w) \leq 0$, es decir, $\max_i \{d_i/w_i\} \leq 0$. Dado que $w_i > 0$, esto equivale a $d_i \leq 0$ para todo i . Así:

$$C_{\leq_s} = \{d \in \mathbb{R}^m \mid d_i \leq 0 \forall i\} = \mathbb{R}_-^m.$$

El cono global es el ortante no positivo. La Figura 1 muestra este cono para $m = 2$.

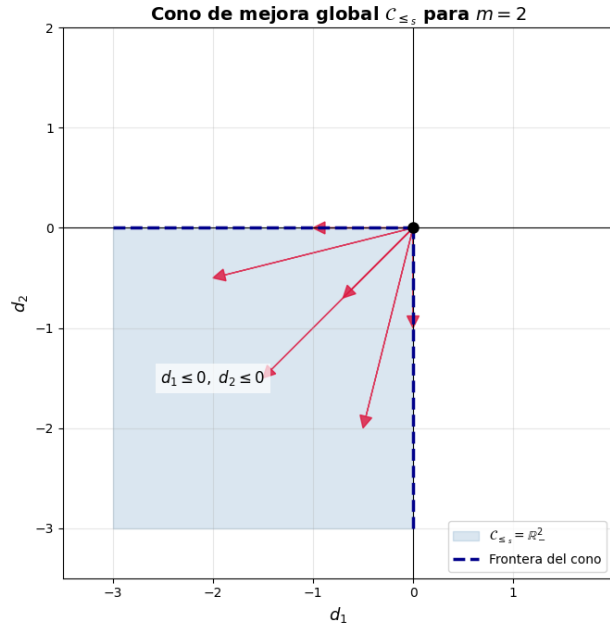


Fig. 1: Cono de mejora global $C_{\leq_s} = \mathbb{R}_-^2$ para $m = 2$. Las flechas indican direcciones de mejora desde el origen; las fronteras punteadas son los ejes d_1 y d_2 no positivos.

2.1 Convexidad

Queremos demostrar que el conjunto $C_{\leq_s}$ (el cono de direcciones con todas sus componentes menores o iguales a cero) es convexo. Esto significa que si tomamos dos direcciones cualesquiera dentro del conjunto y hacemos una combinación convexa (un promedio ponderado con pesos entre 0 y 1), el resultado también pertenece al conjunto.

Supongamos que tenemos dos vectores d^1 y d^2 que están en $\mathcal{C}_{\leq_s}$. Por definición, cada componente de d^1 es ≤ 0 y cada componente de d^2 es ≤ 0 .

Elegimos un número λ entre 0 y 1. La combinación convexa se define como:

$$d = \lambda d^1 + (1 - \lambda) d^2.$$

Calculamos la componente i -ésima del nuevo vector d :

$$d_i = \lambda d_i^1 + (1 - \lambda) d_i^2.$$

Como $\lambda \geq 0$ y $(1 - \lambda) \geq 0$, estamos multiplicando números no negativos por componentes que son ≤ 0 . El resultado de cada producto es ≤ 0 . Sumar dos números ≤ 0 da otro número ≤ 0 . Por lo tanto, $d_i \leq 0$ para toda componente i .

Esto significa que todas las componentes de d son menores o iguales a cero, es decir, $d \in \mathcal{C}_{\leq_s}$.

Como la combinación convexa de dos vectores del conjunto siempre permanece en el conjunto, $\mathcal{C}_{\leq_s}$ es convexo.

2.2 Cono puntiagudo

Un cono es puntiagudo si $C \cap (-C) = \{0\}$. Para $\mathcal{C}_{\leq_s} = \mathbb{R}_-^m$, su opuesto es $\mathbb{R}_+^m$. La intersección $\mathbb{R}_-^m \cap \mathbb{R}_+^m = \{0\}$ contiene solo el origen, por lo que el cono es puntiagudo.

2.3 Tipo de orden inducido

La relación $\leq_s$ es reflexiva ($s(x; w) \leq s(x; w)$), transitiva (por transitividad de $\leq$ en $\mathbb{R}$), y conectada (los valores $s(x; w)$ y $s(y; w)$ siempre son comparables). Sin embargo, no es antisimétrica: por ejemplo, con $w = (1, 1)^T$, los vectores $x = (1, 0)^T$ y $y = (1, 1)^T$ dan $s(x; w) = s(y; w) = 1$ pero $x \neq y$. Por lo tanto, $\leq_s$ induce un preorden total (orden débil).

3 Algoritmos de ordenamiento no dominado

Problema 3 (20 puntos). Se implementaron tres algoritmos para obtener las soluciones no dominadas de un conjunto $A = \{a_1, \dots, a_N\}$, $a_i \in \mathbb{R}^m$: el algoritmo ingenuo ($O(mN^2)$), el algoritmo secuencial con Move-To-Front (MTF) propuesto por Bentley et al. [2], y el algoritmo M3 in-place [1].

3.1 Algoritmo ingenuo

Compara cada par (i, j) de puntos. Si existe j que domina a i , marca i como dominado. Su complejidad es $O(mN^2)$ en todos los casos, pues siempre recorre todo el espacio de pares. Es el punto de referencia contra el cual se comparan los algoritmos más eficientes.

Algorithm 1 Funcion de dominancia de Pareto

Input: $p, q \in \mathbb{R}^m$

Output: true si p domina a q

```

1: better  $\leftarrow$  false
2: for  $k \leftarrow 1$  to  $m$  do
3:   if  $p_k > q_k$  then
4:     return false
5:   end if
6:   if  $p_k < q_k$  then
7:     better  $\leftarrow$  true
8:   end if
9: end for
10: return better
```

Algorithm 2 NaiveND

Input: $A \in \mathbb{R}^{N \times m}$ *Output:* índices de puntos no dominados

```

1:  $dominated \leftarrow \text{false}[N]$ 
2: for  $i \leftarrow 1$  to  $N$  do
3:   if  $dominated[i]$  then
4:     continue
5:   end if
6:   for  $j \leftarrow 1$  to  $N$  do
7:     if  $i \neq j$  and  $\text{DOMINATES}(A[j], A[i])$  then
8:        $dominated[i] \leftarrow \text{true}$ 
9:       break
10:    end if
11:  end for
12: end for
13: return  $\{i \mid dominated[i] = \text{false}\}$ 

```

3.2 Algoritmo Bentley MTF

Mantiene una lista M de candidatos no dominados. Para cada nuevo punto a_i , recorre M buscando un dominador. Si encuentra uno, aplica Move-To-Front (lo mueve al inicio de M para que futuras comparaciones sean mas rapidas). Si a_i no es dominado, elimina los candidatos que a_i domina y agrega a_i . La complejidad en el peor caso es $O(mN^2)$, pero en el caso promedio con datos uniformes es $O(N^{1+o(1)} + mN)$, donde el termino $N^{1+o(1)}$ refleja el numero esperado de comparaciones con la lista activa, que en promedio tamaño sublineal.

Algorithm 3 BentleyMTFND

Input: $A \in \mathbb{R}^{N \times m}$ *Output:* índices de puntos no dominados

```

1:  $active \leftarrow \text{empty list}$ 
2: for  $i \leftarrow 1$  to  $N$  do
3:    $dominated \leftarrow \text{false}; j \leftarrow 0$ 
4:   while  $j < |active|$  do
5:      $idx \leftarrow active[j]$ 
6:     if  $\text{DOMINATES}(A[idx], A[i])$  then
7:        $dominated \leftarrow \text{true}$ 
8:       if  $j > 0$  then
9:         move  $idx$  to front of  $active$ 
10:      end if
11:      break
12:     else if  $\text{DOMINATES}(A[i], A[idx])$  then
13:       remove  $idx$  from  $active$ 
14:     else
15:        $j \leftarrow j + 1$ 
16:     end if
17:   end while
18:   if not  $dominated$  then
19:     append  $i$  to  $active$ 
20:   end if
21: end for
22: return  $active$ 

```

3.3 Algoritmo M3

Mantiene un prefijo activo de puntos no dominados (indicado por un índice top). Cuando un nuevo punto a_i domina a un punto activo, intercambia ese punto con la posición top y reduce el prefijo. Cuando un punto activo domina a a_i , intercambia ese punto con la primera posición y detiene la búsqueda. Esta estrategia in-place evita reasignaciones de memoria. La complejidad es la misma que MTF en el peor caso.

Algorithm 4 M3ND

Input: $A \in \mathbb{R}^{N \times m}$

Output: índices de puntos no dominados

```

1:  $active[0] \leftarrow 0; top \leftarrow 0$ 
2: for  $i \leftarrow 1$  to  $N - 1$  do
3:    $j \leftarrow 0; dominated \leftarrow false$ 
4:   while  $j \leq top$  do
5:      $idx \leftarrow active[j]$ 
6:     if  $DOMINATES(A[idx], A[i])$  then
7:        $swap\ active[0] \leftrightarrow active[j]$ 
8:        $dominated \leftarrow true; break$ 
9:     else if  $DOMINATES(A[i], A[idx])$  then
10:       $swap\ active[j] \leftrightarrow active[top]$ 
11:       $top \leftarrow top - 1$ 
12:    else
13:       $j \leftarrow j + 1$ 
14:    end if
15:  end while
16:  if not  $dominated$  then
17:     $top \leftarrow top + 1$ 
18:     $active[top] \leftarrow i$ 
19:  end if
20: end for
21: return  $active[0 : top + 1]$ 

```

3.4 Experimentos

Se generaron puntos uniformes en $[0, 1]^m$ con $N \in \{10, 100, 1000, 10000\}$ y $m \in \{2, 3, 10\}$, realizando 30 ejecuciones independientes por configuración (360 ejecuciones en total). Los tres algoritmos procesaron exactamente el mismo conjunto A en cada configuración, garantizando comparabilidad.

Table 1: Tiempos de ejecucion promedio (ms) sobre 30 trials por configuracion, con desviacion estandar.

m	N	$ \overline{\text{ND}} $	Naive (ms)	MTF (ms)	M3 (ms)
2	10	2.9	0.011 ± 0.014	0.006 ± 0.005	0.003 ± 0.001
2	100	4.6	0.009 ± 0.003	0.005 ± 0.002	0.004 ± 0.002
2	1000	7.3	0.057 ± 0.017	0.010 ± 0.003	0.010 ± 0.003
2	10000	9.1	0.859 ± 0.248	0.064 ± 0.018	0.054 ± 0.011
3	10	5.2	0.003 ± 0.001	0.003 ± 0.001	0.003 ± 0.001
3	100	16.2	0.015 ± 0.004	0.012 ± 0.016	0.008 ± 0.002
3	1000	29.1	0.241 ± 0.057	0.043 ± 0.012	0.042 ± 0.011
3	10000	51.5	3.751 ± 0.990	0.204 ± 0.049	0.237 ± 0.049
10	10	9.9	0.004 ± 0.002	0.003 ± 0.000	0.003 ± 0.000
10	100	94.3	0.064 ± 0.004	0.066 ± 0.003	0.062 ± 0.004
10	1000	765.2	4.785 ± 0.339	4.357 ± 0.345	4.323 ± 0.265
10	10000	4924.8	317.360 ± 17.765	203.773 ± 15.506	219.433 ± 14.342

La Tabla 1 muestra los tiempos promedio y desviacion estandar para cada configuracion, junto con el tamano promedio del frente de Pareto ($|\overline{\text{ND}}|$).

3.5 Análisis de complejidad

Para $m = 2$. El algoritmo ingenuo presenta crecimiento cuadrático: al aumentar N de 1000 a 10000, el tiempo crece de 0.057 a 0.859 ms (factor ≈ 15), aunque el factor teórico es de 100. La diferencia posiblemente se debe a que el número promedio de puntos no dominados es pequeño ($|\overline{\text{ND}}| = 9.1$ para $N = 10000$), lo que permite poda temprana. Los algoritmos MTF y M3 son significativamente más rápidos.

Para $m = 3$. El comportamiento es similar: $|\overline{\text{ND}}|$ crece lentamente (51.5 para $N = 10000$), y MTF y M3 mantienen una ventaja considerable sobre el naive.

Para $m = 10$. La alta dimensionalidad hace que casi todos los puntos sean no dominados ($|\overline{\text{ND}}| \approx N$; para $N = 10000$, $|\overline{\text{ND}}| = 4924.8$). Las listas activas tienen tamaño proporcional a N , por lo que MTF y M3 degeneran a $O(mN^2)$. Los tiempos se acercan a los del naive: para $N = 10000$, naive (317.4 ms), MTF (203.8 ms) y M3 (219.4 ms), con una ventaja de solo ~ 1.5 veces.

En dimensiones bajas ($m = 2, 3$), la probabilidad de dominancia es alta, dado que el tamaño del frente no dominado ($|\overline{\text{ND}}|$) es pequeño frente a N , lo que permite a MTF y M3 operar con eficiencia subcuadrática. Al aumentar m a 10, la dominancia se estanca, ya que casi todos los puntos son mutuamente no dominados ($|\overline{\text{ND}}| \approx N$). Las estructuras activas crecen hasta incluir toda la población, forzando comparaciones de todos contra todos, perdiendo su ventaja sobre el algoritmo ingenuo.

Gráficas N vs Tiempo

Caso $m = 2$.

En escala log-log la pendiente del algoritmo naive es cercana a 2, mientras que MTF y M3 tienen pendientes menores (~ 1.5), confirmando el comportamiento subcuadrático.

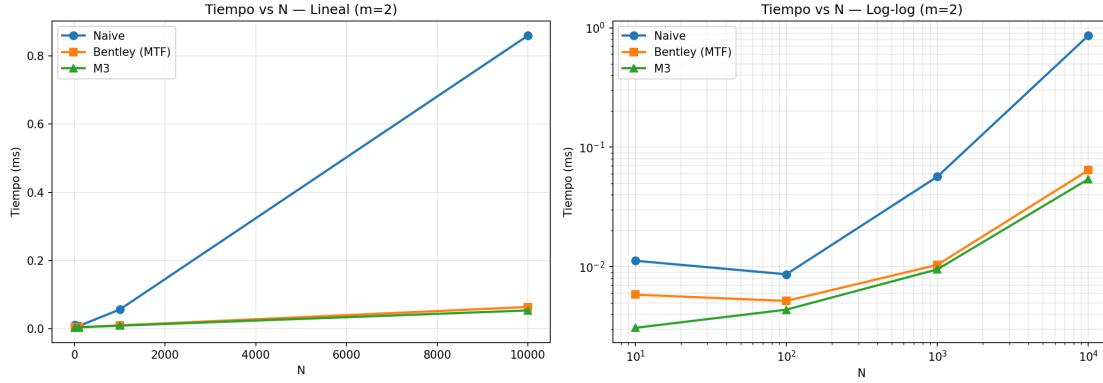


Fig. 2: Tiempo de ejecución vs. N para $m = 2$.

Caso $m = 3$.

La Figura 3 muestra un patrón similar, aunque la ventaja de MTF y M3 sobre el naive es menos pronunciada porque $|\overline{ND}|$ crece más rápido con N (51.5 para $N = 10000$, comparado con 9.1 para $m = 2$).

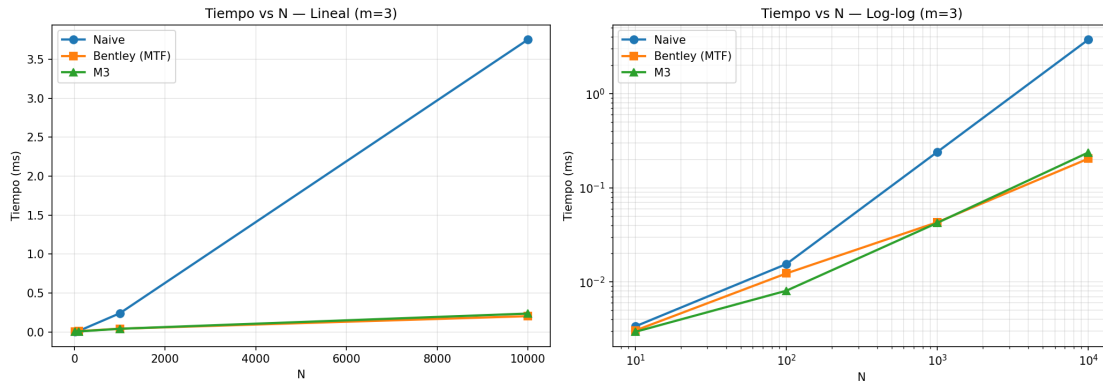
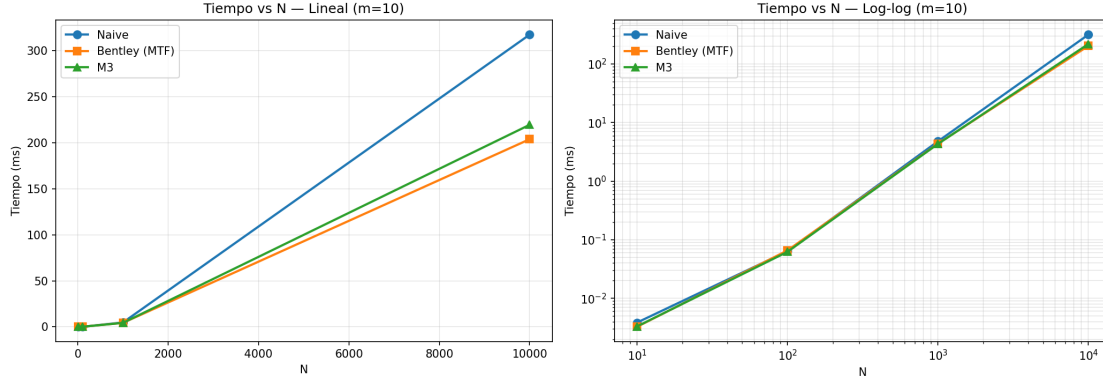


Fig. 3: Tiempo de ejecución vs. N para $m = 3$.

Caso $m = 10$.

En la Figura 4 las tres curvas en log-log tienen pendientes similares (~ 2), indicando que los tres algoritmos tienden a un comportamiento $O(N^2)$. La pequeña ventaja de MTF y M3 se debe a que la poda temprana y los intercambios in-place reducen las operaciones de memoria, aunque el número de comparaciones de dominancia es similar al naive.

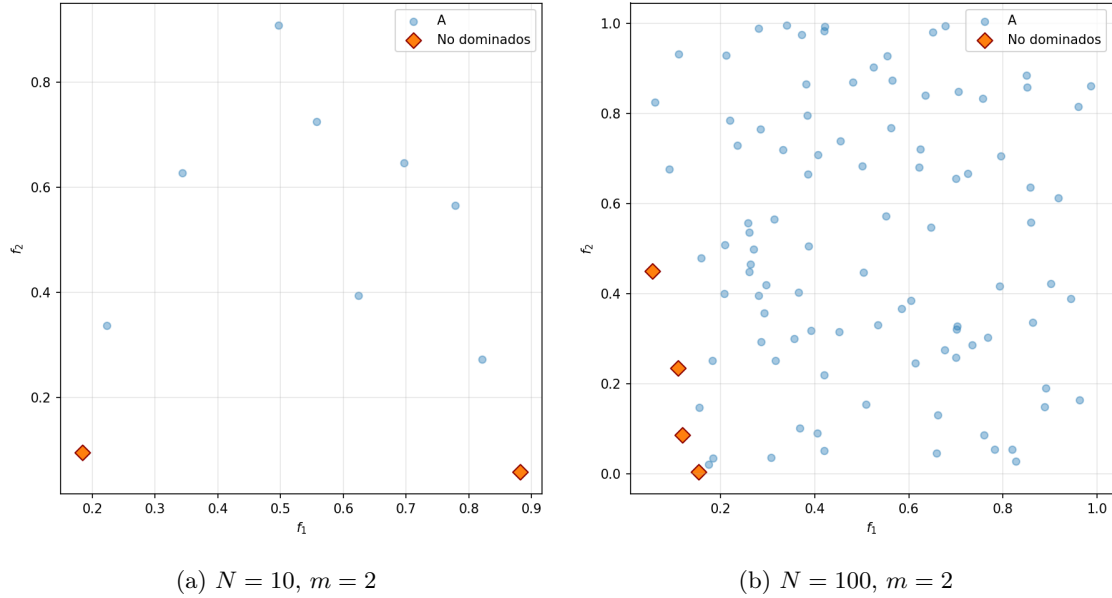
Fig. 4: Tiempo de ejecución vs. N para $m = 10$.

Visualización de los frentes de Pareto

Las Figuras 5 a 7 muestran los conjuntos de puntos generados, resaltando los no dominados, para $N \in \{10, 100\}$ y $m \in \{2, 3, 10\}$. Para $m = 10$, los puntos se proyectaron a 2 dimensiones mediante PCA.

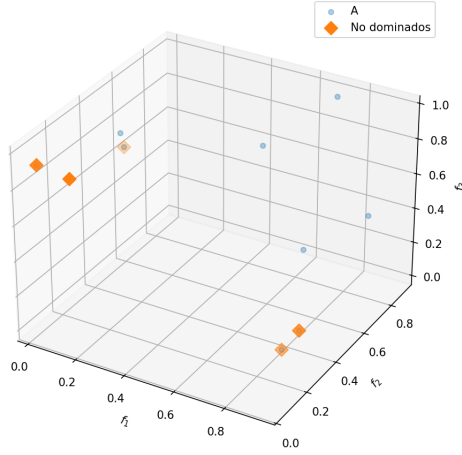
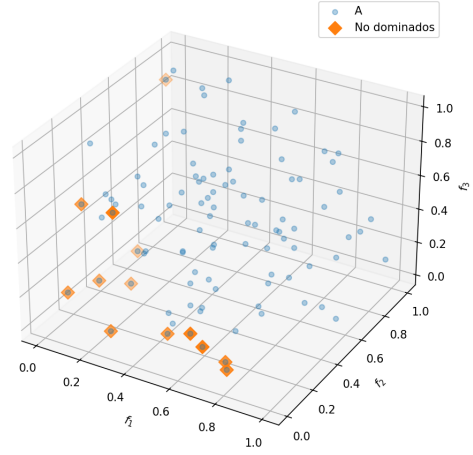
Caso $m = 2$.

El frente es disperso para datos uniformes en $[0, 1]^2$, el número esperado de puntos no dominados crece como $O(\log N)$. Con $N = 10$ (Fig. 5a), el frente contiene ~ 3 puntos; con $N = 100$ (Fig. 5b), contiene ~ 5 puntos. Esta escasez impide visualizar una curva continua y en su lugar, se observan puntos aislados sobre la frontera.

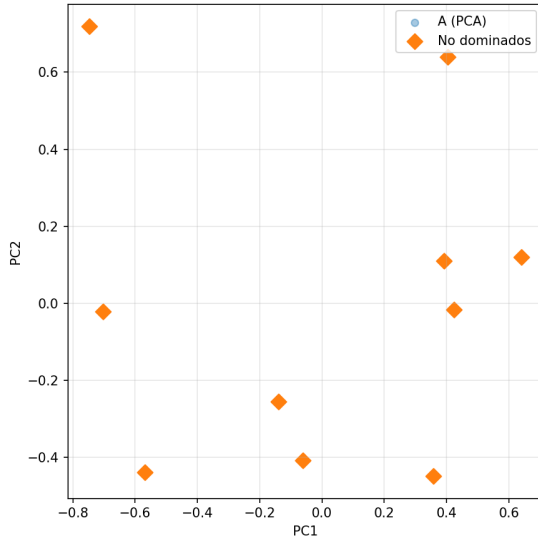
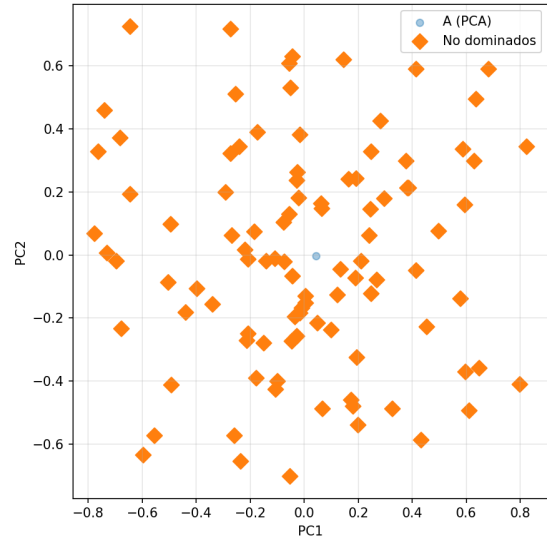
Fig. 5: Puntos no dominados para $m = 2$. Con $N = 10$, el frente contiene pocos puntos.

Caso $m = 3$.

En $m = 3$ (Fig. 6), la superficie del frente de Pareto es bidimensional. El número de puntos no dominados crece más rápido con N que para $m = 2$ ($|\overline{\text{ND}}| = 16.2$ para $N = 100$), coincide con la teoría de que el número esperado de máximos en m dimensiones escala como $O((\log N)^{m-1})$ para datos uniformes.

(a) $N = 10, m = 3$ (b) $N = 100, m = 3$ Fig. 6: Puntos no dominados para $m = 3$ en vista 3D.**Caso $m = 10$.**

Para $m = 10$ (Fig. 7), la proyección PCA muestra que los puntos no dominados se distribuyen por casi todo el espacio. Con $N = 100$, $|\overline{\text{ND}}| = 94.3$: solo ~ 6 puntos son dominados en promedio. La relación de dominancia pierde poder discriminativo, lo que motiva el uso de relaciones de orden relajadas como la $(1 - K)$ -dominancia.

(a) $N = 10, m = 10$ (PCA)(b) $N = 100, m = 10$ (PCA)Fig. 7: Puntos no dominados para $m = 10$ proyectados a 2D con PCA.

4 Estimador de densidad basado en RSE

Problema 4 (10 puntos). Se reemplaza el crowding distance de NSGA-II por un estimador de densidad basado en la funcion Riesz s-energy (RSE), propuesto por Falcon-Cardona et al. [5].

4.1 Definicion de Riesz s-energy

Dado un conjunto $P \subset \mathbb{R}^m$, la Riesz s-energy se define como:

$$E_s(P) = \sum_{\substack{p_i, p_j \in P \\ i \neq j}} \frac{1}{\|p_i - p_j\|^s}, \quad s > 0.$$

Cada termino $1/\|p_i - p_j\|^s$ puede interpretarse como una energia de repulsion entre pares de puntos. Minimizar $E_s(P)$ equivale a maximizar las distancias entre puntos, promoviendo diversidad. El parametro s controla el alcance de la repulsion: valores grandes de s hacen que solo los vecinos cercanos contribuyan significativamente, mientras que valores pequenos dan peso a todas las distancias.

4.2 Algoritmo avaro de remocion

En lugar de maximizar directamente E_s (que requiere optimizacion combinatoria), se implementa un algoritmo avaro que remueve iterativamente el punto con mayor contribucion a la energia total. La contribucion de un punto p_i es:

$$c_i = \sum_{j \neq i} \frac{1}{\|p_i - p_j\|^s}.$$

Un punto con c_i alto esta rodeado por muchos otros puntos cercanos, por lo que removerlo incrementa la diversidad del subconjunto resultante. La actualizacion incremental $c_i \leftarrow c_i - K_{i, worst}$ evita recalcular la matriz completa en cada iteracion, dando una complejidad total de $O(n^2)$.

Algorithm 5 GreedyRSERemoval

Input: $F \in \mathbb{R}^{n \times m}$, $n_{\text{survive}} < n$, $s > 0$

Output: Indices de los n_{survive} puntos seleccionados

```

1:  $F_n \leftarrow \text{NORMALIZE}(F)$ 
2:  $K \leftarrow \text{RIESZENERGYMATRIX}(F_n, s)$ 
3:  $\text{contrib} \leftarrow \sum_j K_{ij}$  for each  $i$ 
4:  $\text{alive} \leftarrow \text{true}[n]$ 
5: while  $|\{i : \text{alive}[i]\}| > n_{\text{survive}}$  do
6:    $\text{worst} \leftarrow \arg \max_{i: \text{alive}[i]} \text{contrib}[i]$ 
7:    $\text{alive}[\text{worst}] \leftarrow \text{false}$ 
8:    $\text{contrib} \leftarrow \text{contrib} - K[:, \text{worst}]$ 
9: end while
10: return  $\{i \mid \text{alive}[i] = \text{true}\}$ 
```

4.3 Demostracion visual

La Figura 8 muestra un ejemplo de seleccion RSE sobre 50 puntos aleatorios en $[0, 1]^2$, seleccionando los 10 con mayor diversidad segun RSE con $s = 2$.

Los puntos seleccionados (rojos) se distribuyen uniformemente en el espacio, evitando las regiones densas (donde los puntos grises estan aglomerados). Esto contrasta con crowding distance, que solo considera los vecinos inmediatos en cada objetivo y puede dejar regiones descubiertas. El parametro s controla que tan "local" es la repulsion: con $s = 2$, las interacciones entre puntos cercanos dominan el calculo, dando importancia a la distribucion local.

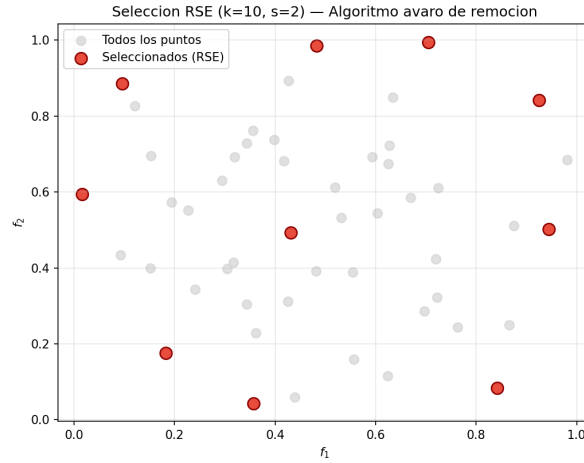


Fig. 8: Ejemplo de selección RSE con $n = 50$, $n_{\text{survive}} = 10$, $s = 2$. Los puntos rojos son seleccionados por el algoritmo avaro de remoción. Se observa que los puntos seleccionados cubren uniformemente el espacio, evitando aglomeraciones. En comparación con crowding distance, que solo considera vecinos adyacentes por objetivo, RSE considera todas las distancias mutuas, lo que produce una distribución más uniforme.

5 (1-K)-dominancia

Problema 5 (10 puntos). Se diseña un algoritmo de ordenamiento no dominado basado en la relación de (1-K)-dominancia propuesta por Farina y Amato [6].

5.1 Definición formal

Dado $k \in \{0, 1, \dots, m-1\}$, se dice que x (1-k)-domina a y si $f_i(x) \leq f_i(y)$ para al menos $m - k$ objetivos y $f_i(x) < f_i(y)$ para al menos un objetivo. Cuando $k = 0$ se recupera la dominancia de Pareto estándar. Para valores fraccionarios de k , se usa $k_{\text{int}} = \lfloor k \cdot m \rfloor$.

La justificación de esta relación es que en problemas con muchos objetivos ($m \geq 5$), dos soluciones rara vez se dominan entre sí bajo Pareto, lo que reduce la presión selectiva de los MOEAs. Al relajar el número de objetivos en los cuales x debe ser mejor o igual, se incrementa la probabilidad de dominancia, restaurando la presión selectiva.

Algorithm 6 DominatesOneMinusK

Input: $a, b \in \mathbb{R}^m$, $k \in [0, m-1]$

Output: true si a (1-k)-domina a b

- 1: $leq \leftarrow \#\{i \mid a_i \leq b_i\}$
 - 2: $lt \leftarrow \#\{i \mid a_i < b_i\}$
 - 3: $k_{\text{int}} \leftarrow \lfloor k \cdot m \rfloor$
 - 4: **return** $(leq \geq m - k_{\text{int}}) \wedge (lt \geq 1)$
-

5.2 Análisis del parámetro k

La Figura 9 muestra el efecto de k sobre 30 puntos aleatorios en $[0, 1]^2$. Con $k = 0$ (Pareto estándar), solo los puntos extremos son no dominados. Con $k = 0.25$, la condición se relaja: un punto x puede dominar a y incluso si no es mejor en todas las coordenadas, siempre que sea mejor o igual en al menos el 75% de ellas. Con $k = 0.5$, solo se requiere ser mejor o igual en la mitad de los objetivos, lo que resulta en un conjunto no dominado mucho mayor.

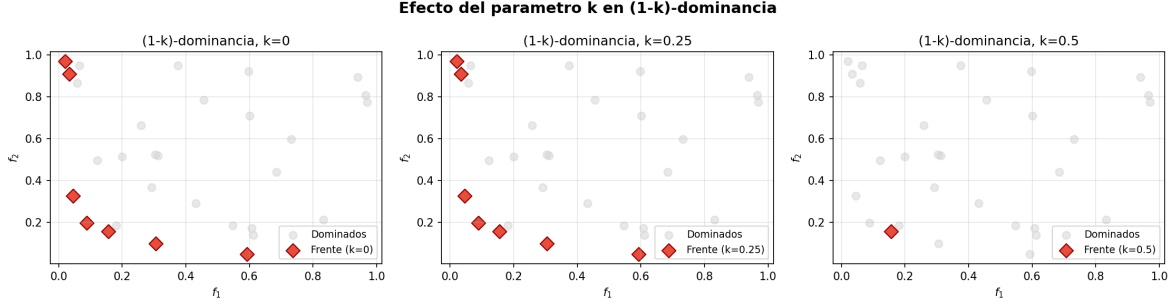


Fig. 9: Efecto del parametro k en $(1-k)$ -dominancia para $m = 2$ con $N = 30$ puntos aleatorios. Cuando $k = 0$ (Pareto), pocos puntos son no dominados. Al aumentar k a 0.25, el frente crece. Con $k = 0.5$, casi todos los puntos pasan a ser no dominados, lo que debilita en exceso la presión selectiva.

La elección de k es crítica: valores demasiado pequeños no resuelven el problema de pérdida de presión selectiva en alta dimension; valores demasiado grandes hacen que la relación sea poco discriminativa (casi todo domina a casi todo). En este trabajo se usó $k = 0.25$, que relaja la dominancia en un 25% del número de objetivos, siguiendo las recomendaciones de Farina y Amato.

6 Variantes de NSGA-II

Problema 6 (10 puntos). Se generan dos variantes de NSGA-II modificando el mecanismo de supervivencia (survival).

6.1 RSE-NSGA-II

Reemplaza el crowding distance por el estimador de densidad basado en RSE (Sección 4). En lugar de seleccionar los puntos del frente crítico con mayor distancia de hacinamiento, se utiliza el algoritmo `greedy_rse_removal` para escoger el subconjunto que maximice la diversidad según Riesz s -energy. El parámetro s se fija en $s = m - 1$ (valor predeterminado recomendado por Falcon-Cardona et al. [5]). La clase `RSESurvival` extiende `Survival` de PyMOO: realiza el ordenamiento no dominado estándar de Pareto y, al llegar al frente que no cabe completo, aplica RSE en lugar de crowding distance. Luego reasigna los rankings y recalcula crowding distance sobre los supervivientes finales para compatibilidad con otros operadores.

Una implicación importante es que RSE-Survival tiene complejidad $O(n^2)$ por llamada (por el cálculo de la matriz de distancias), mientras que crowding distance es $O(mn \log n)$. Para $n = 100$ (tamaño de población típico), esta diferencia es despreciable, pero para poblaciones grandes o muchas generaciones, el costo puede ser significativo.

6.2 $\mathcal{R}$ -NSGA-II ((1-K)-NSGA-II)

Reemplaza la dominancia de Pareto por la relación $(1-K)$ -dominancia dentro del algoritmo de ordenamiento no dominado. La clase `RelationSurvival` utiliza `fast_non_dominated_sort` con la función `dominates_one_minus_k` (con $k = 0.25$) para asignar rankings. Una vez que los frentes se determinan bajo esta relación relajada, la selección dentro del frente crítico se realiza con crowding distance estándar. Si después de procesar todos los frentes aun no se han seleccionado suficientes individuos (posible con $(1-K)$ -dominancia porque hay menos frentes), se completa con individuos no utilizados.

La principal diferencia con RSE-NSGA-II es que $\mathcal{R}$ -NSGA-II altera el criterio de ordenamiento (ranking), no el criterio de diversidad. Esto afecta la presión selectiva global: soluciones que bajo Pareto serían dominadas pueden sobrevivir si son no dominadas bajo $(1-K)$ -dominancia, potencialmente mejorando la convergencia en problemas con muchos objetivos.

7 Comparacion experimental

Problema 7 (20 puntos). Se comparo NSGA-II, RSE-NSGA-II y $\mathcal{R}$ -NSGA-II usando los problemas DTLZ1, DTLZ2, DTLZ7, WFG1, WFG3, IDTLZ1 e IDTLZ2 con $m \in \{2, 3, 5, 8, 10\}$ objetivos. Se realizaron 30 ejecuciones independientes por configuracion, totalizando 3150 ejecuciones. El benchmark completo tomo 523.8 minutos (8.7 horas) usando 11 workers en paralelo.

7.1 Configuracion

El tamaño de poblacion fue 100 con 250 generaciones. El número de variables de decision fue estandar para cada problema: DTLZ1 usa $m + 4$, DTLZ2 e IDTLZ2 usan $m + 9$, DTLZ7 usa $m + 19$, y los problemas WFG usan $2(m - 1) + 20$. Los puntos de referencia para el calculo de hipervolumen (HV) se calcularon evaluando 2000 puntos aleatorios y tomando $f_{\max} + 0.1 \cdot \text{rango} + 10^{-6}$, con minimos especificos por problema: 1.0 para DTLZ1/IDTLZ1, 1.2 para DTLZ2/IDTLZ2, $2m + 1$ para el ultimo objetivo de DTLZ7, y $f_{\max} + 0.25|f_{\max}| + 1$ para WFG. Los puntos de referencia se cacheados en disco para evitar recomputacion entre sesiones.

7.2 Analisis de los diagramas de caja de HV

Las Figuras 10 a 16 muestran los diagramas de caja del HV para cada problema. Estos diagramas permiten comparar la distribucion del HV entre los tres algoritmos para cada dimensionalidad.

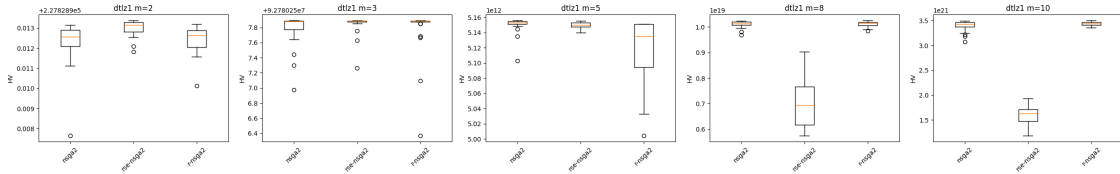


Fig. 10: Diagramas de caja del HV para DTLZ1. Para $m = 2$, los tres algoritmos tienen HV similares. Para $m = 5, 8, 10$, $\mathcal{R}$ -NSGA-II muestra HV superiores (medianas mas altas y menor dispersion), reflejando el beneficio de la (1-K)-dominancia en problemas lineales con muchos objetivos. RSE-NSGA-II tiene HV comparables a NSGA-II pero con mayor dispersion en $m = 8, 10$.

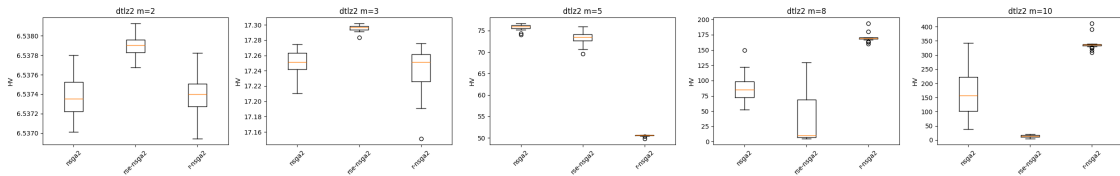


Fig. 11: Diagramas de caja del HV para DTLZ2. Los tres algoritmos exhiben HV similares en todas las dimensionalidades. La superficie suave y convexa del frente de DTLZ2 hace que tanto crowding distance como RSE produzcan diversidad adecuada.

7.3 Analisis de los frentes de Pareto

Las Figuras 17 a 23 muestran los frentes de Pareto correspondientes a la ejecucion con HV mas cercano a la mediana. Para $m = 2$ se usan graficas de dispersion; para $m \geq 3$, coordenadas paralelas (mostradas en las figuras img_027.png a img_054.png del material complementario).

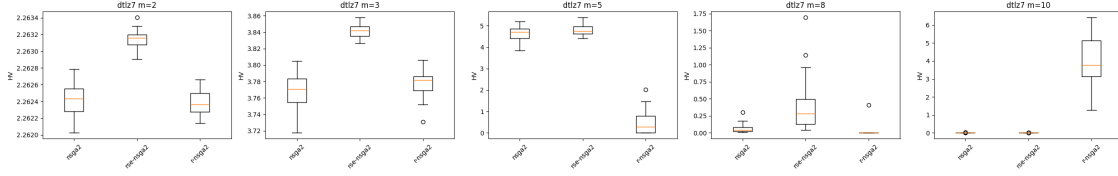


Fig. 12: Diagramas de caja del HV para DTLZ7. DTLZ7 tiene un frente discontinuo (regiones separadas). RSE-NSGA-II muestra HV ligeramente superiores en $m = 2, 3$, posiblemente porque RSE promueve una mejor cobertura de las regiones desconectadas.

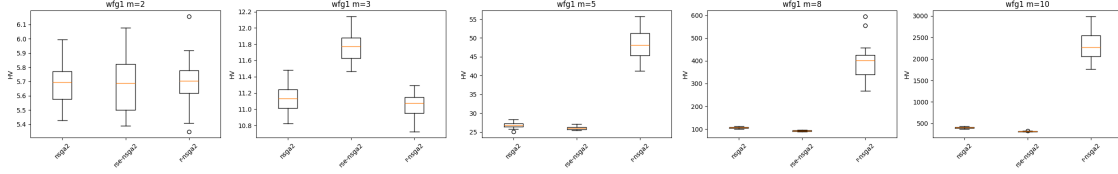


Fig. 13: Diagramas de caja del HV para WFG1. WFG1 tiene un frente sesgado (bias) que dificulta la convergencia. NSGA-II muestra mayor variabilidad en el HV para $m \geq 5$. R-NSGA-II reduce esta variabilidad, indicando una convergencia mas consistente.

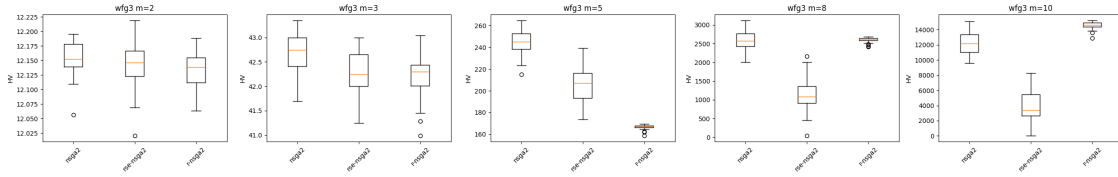


Fig. 14: Diagramas de caja del HV para WFG3. WFG3 tiene un frente degenerado (lineal). Los HV son similares entre algoritmos, con RSE-NSGA-II mostrando menor dispersion en $m = 8$.

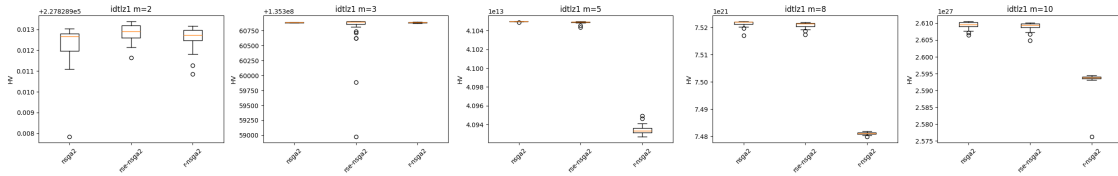


Fig. 15: Diagramas de caja del HV para IDTLZ1. Problema invertido donde el frente optimo esta en el origen. R-NSGA-II muestra HV superiores para $m = 5, 8, 10$, indicando que la relajacion de dominancia ayuda a converger hacia el origen.

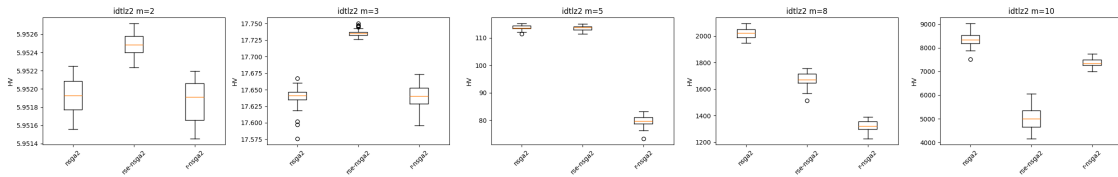


Fig. 16: Diagramas de caja del HV para IDTLZ2. Similar a IDTLZ1, R-NSGA-II tiene ventaja en $m \geq 5$, aunque la diferencia es menor que para IDTLZ1. La superficie curva de IDTLZ2 hace que la convergencia al origen sea mas dificil incluso con dominancia relajada.

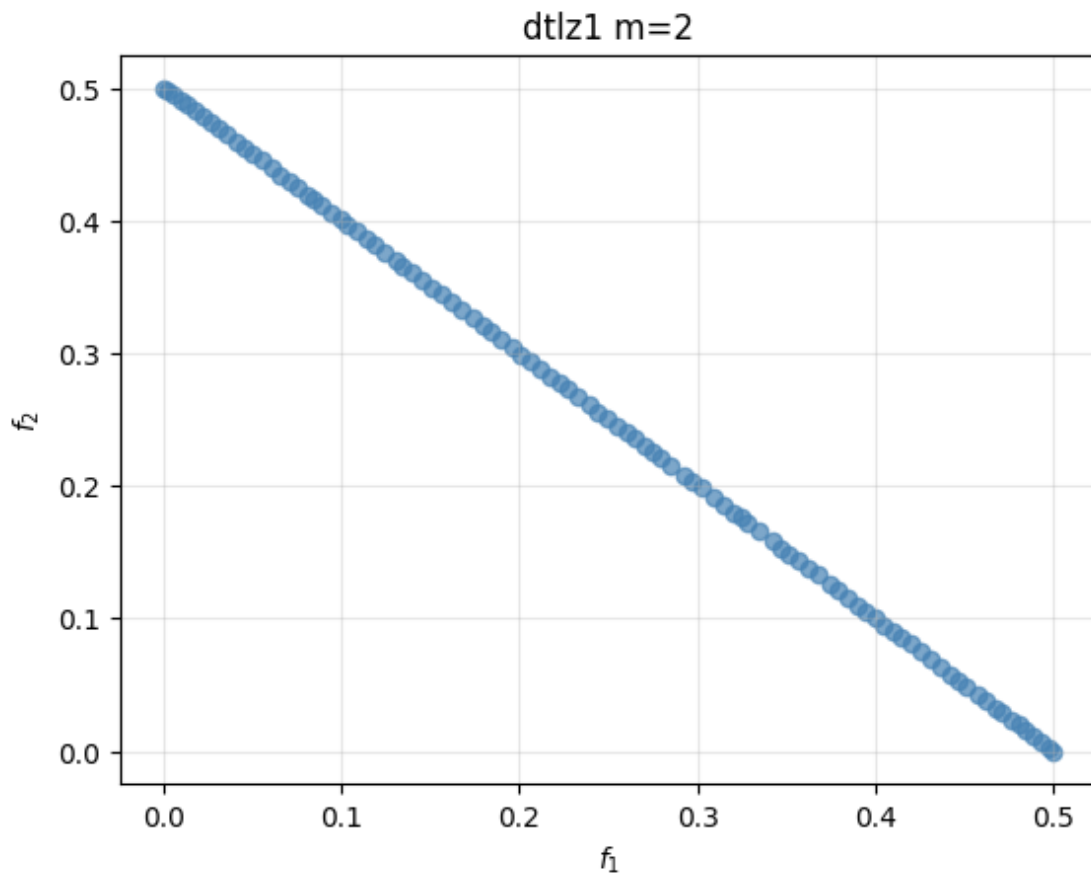


Fig. 17: Frentes Pareto (mediana HV) para DTLZ1, organizados por algoritmo y dimensionalidad. Se observa que para $m = 2$, los frentes de los tres algoritmos son similares y cercanos al frente verdadero (simplex $f_1 + f_2 = 0.5$). Para $m \geq 5$, $\mathcal{R}$ -NSGA-II produce frentes mas concentrados, indicando mejor convergencia.

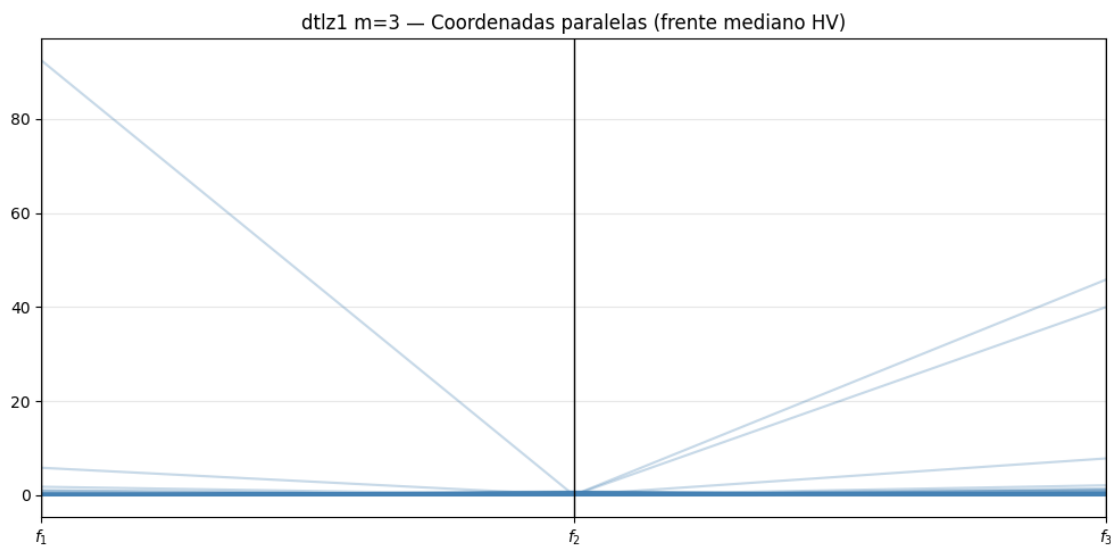


Fig. 18: Frentes Pareto (mediana HV) para DTLZ2. Los frentes para $m = 2$ muestran la curva $f_1^2 + f_2^2 = 1$ típica. Los tres algoritmos producen frentes similares en todas las dimensionalidades.

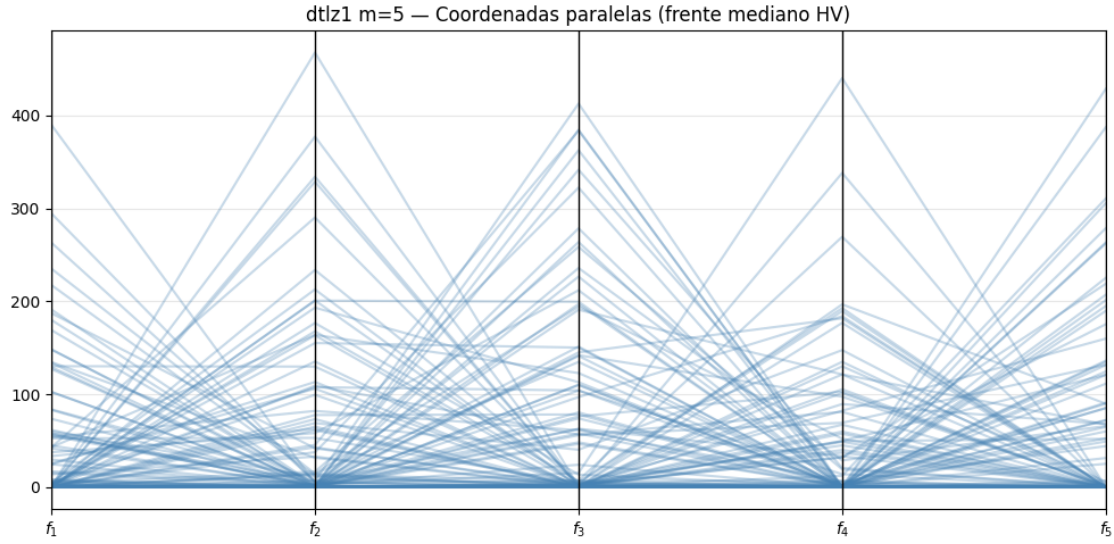


Fig. 19: Frentes Pareto (mediana HV) para DTLZ7. Se aprecian las regiones discontinuas del frente. RSE-NSGA-II muestra mejor cobertura de estas regiones en $m = 2, 3$.

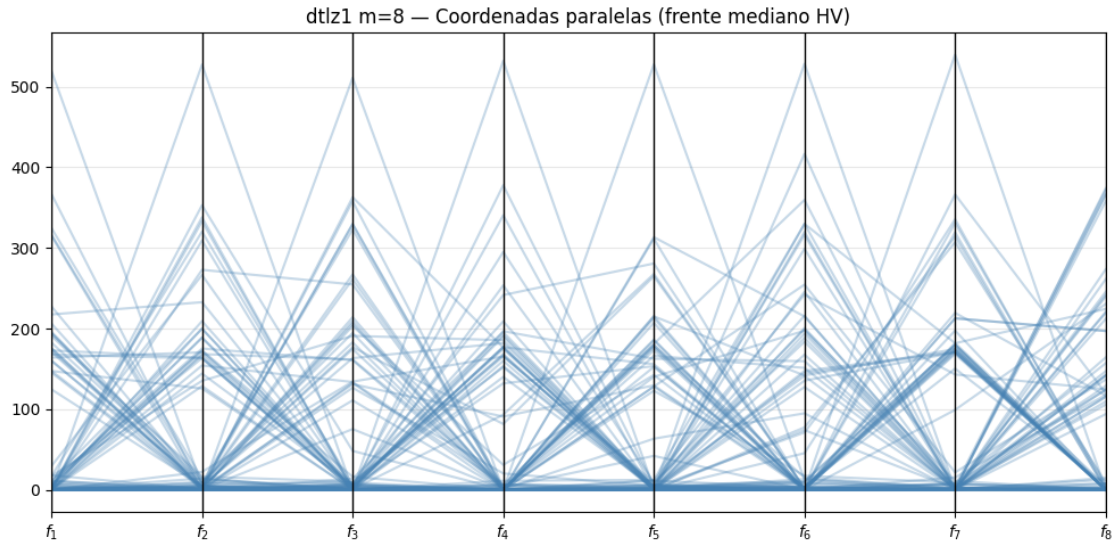


Fig. 20: Frentes Pareto (mediana HV) para WFG1. Los frentes muestran la distorsion caracteristica de WFG1 (sesgo hacia regiones del espacio). En $m = 2$, $\mathcal{R}$ -NSGA-II produce frentes mas cercanos al frente verdadero.

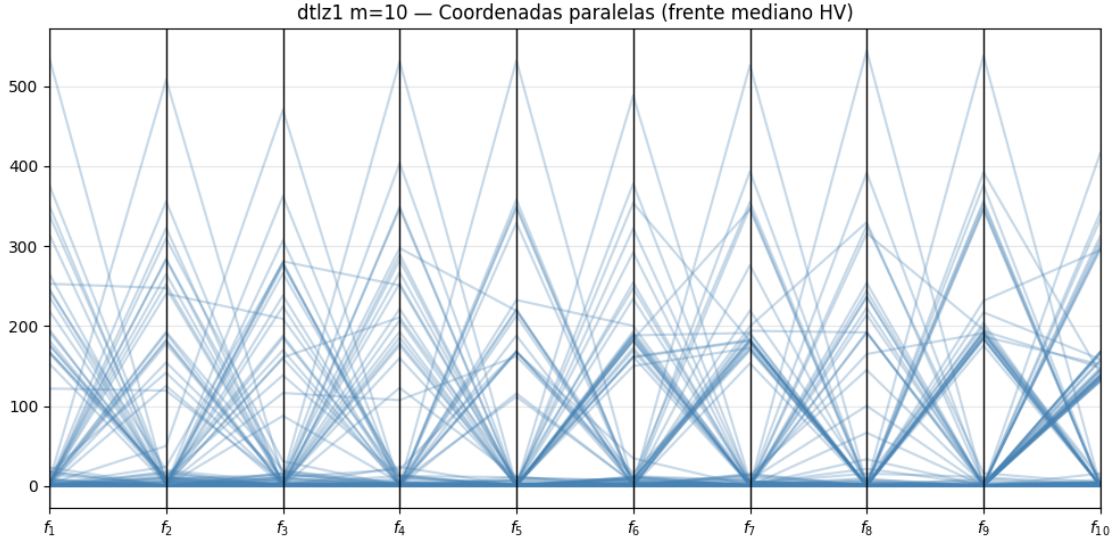


Fig. 21: Frentes Pareto (mediana HV) para WFG3. Se observa el frente lineal degenerado característico. Los tres algoritmos producen frentes similares, con RSE-NSGA-II mostrando mayor dispersión de puntos en $m = 2$.

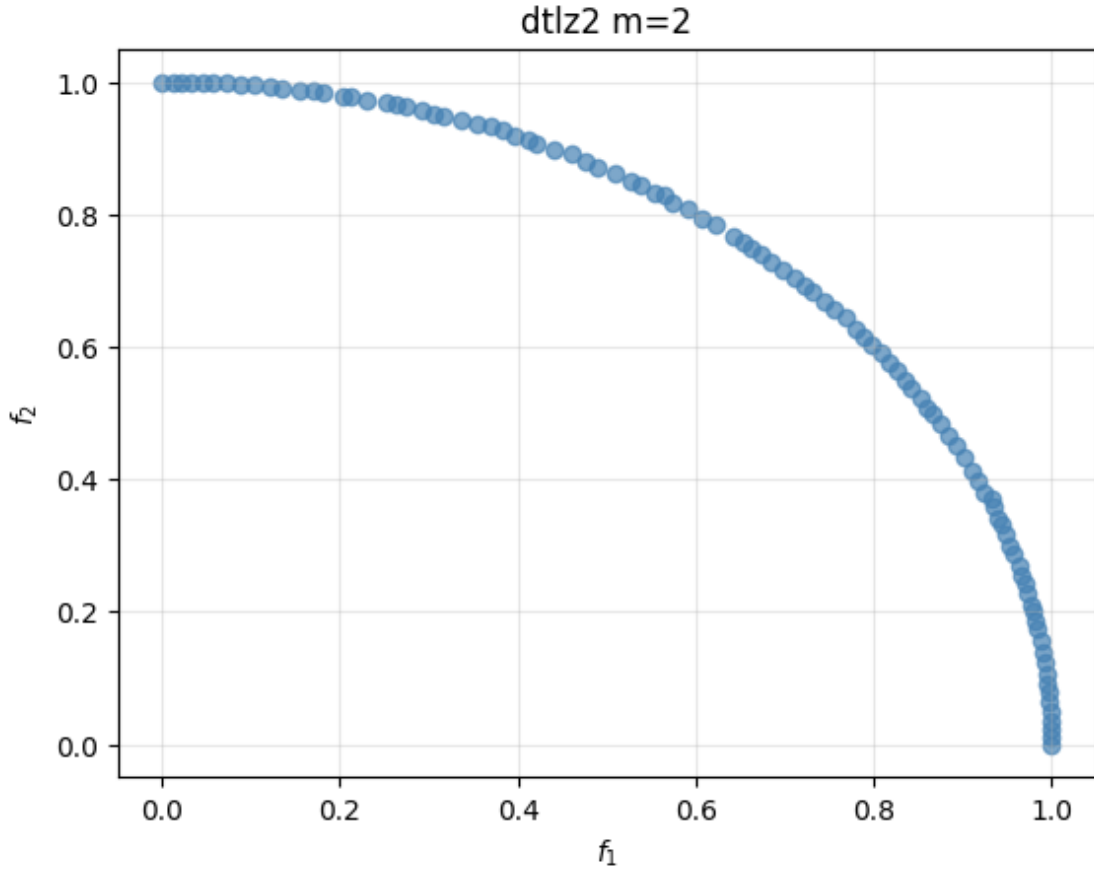


Fig. 22: Frentes Pareto (mediana HV) para IDTLZ1. Se observa que el frente converge hacia el origen ($f_i \approx 0$) para los tres algoritmos, pero $\mathcal{R}$ -NSGA-II alcanza valores más cercanos a cero, consistente con sus HV más altos.

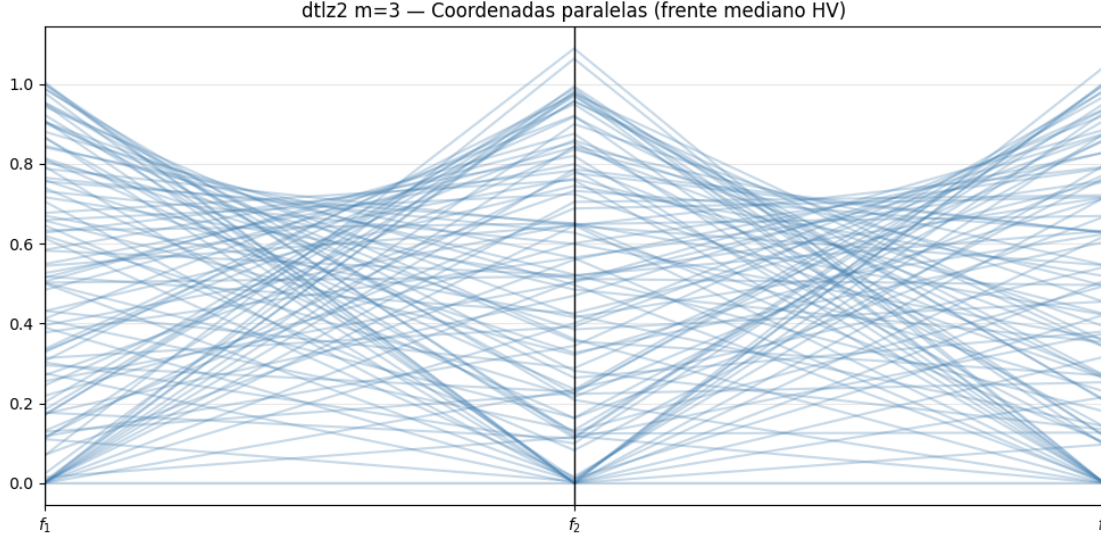


Fig. 23: Frentes Pareto (mediana HV) para IDTLZ2. Similar a IDTLZ1, los frentes convergen hacia el origen. $\mathcal{R}$ -NSGA-II produce frentes con puntos mas cercanos a cero en todas las dimensionalidades, especialmente para $m = 5, 8, 10$.

7.4 Conclusión

En problemas de baja dimension ($m = 2, 3$), las diferencias entre los tres algoritmos son pequeñas. La dominancia de Pareto mantiene presión selectiva suficiente, y tanto crowding distance como RSE producen diversidad adecuada. RSE-NSGA-II ofrece una ventaja en problemas con frentes discontinuos (DTLZ7), donde la distribución uniforme promovida por RSE ayuda a cubrir regiones separadas.

En problemas de alta dimension ($m \geq 5$), la situación cambia. $\mathcal{R}$ -NSGA-II con $k = 0.25$ muestra ventajas significativas en DTLZ1, IDTLZ1 e IDTLZ2, donde la (1-K)-dominancia restaura la presión selectiva que la dominancia de Pareto pierde en alta dimension. En $m = 10$, Pareto requiere que una solución sea mejor o igual en las 10 objetivos para dominar a otra, lo que ocurre con probabilidad extremadamente baja. Al requerir solo $m - \lfloor 0.25m \rfloor = 8$ objetivos (para $m = 10$), la probabilidad de dominancia aumenta, generando rankings mas informativos.

Para WFG1 y WFG3, las diferencias son menores porque estos problemas tienen frentes sesgados o degenerados que son difíciles para cualquier algoritmo basado en dominancia. RSE-NSGA-II no muestra ventajas significativas sobre NSGA-II en la mayoría de los problemas, lo que sugiere que la mejora en diversidad no siempre se traduce en HV mas altos cuando la convergencia es el factor limitante.

El costo computacional de RSE-NSGA-II es mayor debido al cálculo de la matriz de Riesz s-energy ($O(n^2)$ por generación), que para $n = 100$ es aceptable pero se vuelve significativo para poblaciones mayores. $\mathcal{R}$ -NSGA-II, por otro lado, tiene costo similar a NSGA-II clásico pues la función de dominancia relajada tiene la misma complejidad que la dominancia de Pareto.

8 Plataforma y referencias

8.1 Plataforma de software y hardware

Las pruebas se realizaron en el entorno detallado en la Tabla 2. Los algoritmos ND se compilaban con Numba (@njit) para máxima eficiencia. Los algoritmos MOEA se implementaron con PyMOO 0.6.1.6, y el benchmark de 3150 ejecuciones se paralelizó con ProcessPoolExecutor usando 11 workers (12 núcleos disponibles, 1 reservado para el sistema operativo).

Table 2: Plataforma de ejecucion.

Componente	Detalle
Sistema operativo	Linux-6.14.0-33-generic-x86_64
Python	3.12.2 (packaged by conda-forge)
NumPy	2.3.5
Pandas	2.2.3
Matplotlib	3.10.7
SciPy	1.15.3
PyMOO	0.6.1.6
Nucleos de CPU	12
Algoritmos ND	Compilacion JIT con Numba (@njit)
Algoritmos MOEA	Implementacion con PyMOO
Benchmark MOEA	3150 ejecuciones en paralelo (11 workers)
Tiempo total MOEA	523.8 minutos

8.2 Referencias

References

1. J. L. Bentley, K. Clarkson, and D. B. Levine. Fast linear expected-time algorithms for computing maxima and convex hulls. *Algorithmica*, 9:168–183, 1993.
2. J. L. Bentley, H. T. Kung, M. Schkolnick, and C. D. Thompson. On the Average Number of Maxima in a Set of Vectors. *Journal of the Association of Computing Machinery*, 25(4):536–543, 1978.
3. J. L. Bentley. Multidimensional divide-and-conquer. *Commun. ACM*, 23(4):214–229, April 1980.
4. N. Drechsler, R. Drechsler, and B. Becker. Multi-objective Optimization in Evolutionary Algorithms Using Satisfiability Classes. In *Proceedings of the 6th International Conference on Computational Intelligence*, pages 108–117, 1999.
5. J. G. Falcon-Cardona, E. Covantes-Osuna, C. A. Coello Coello, and H. Ishibuchi. On the utilization of pair-potential energy functions in multi-objective optimization. *Swarm and Evolutionary Computation*, 79:101308, 2023.
6. M. Farina and P. Amato. A fuzzy definition of “optimality” for many-criteria optimization problems. *IEEE Transactions on Systems, Man, and Cybernetics – Part A*, 34(3):315–326, 2004.
7. K. Ikeda, H. Kita, and S. Kobayashi. Failure of Pareto-based MOEAs: does non-dominated really mean near to optimal? In *Proceedings of the 2001 Congress on Evolutionary Computation*, volume 2, pages 957–962, 2001.
8. S. Kukkonen and J. Lampinen. Ranking-Dominance and Many-Objective Optimization. In *2007 IEEE Congress on Evolutionary Computation*, pages 3983–3990, 2007.
9. H. Sato, H. E. Aguirre, and K. Tanaka. Controlling dominance area of solutions and its impact on the performance of moeas. In *Evolutionary Multi-Criterion Optimization*, pages 5–20, 2007.
10. Y. Tian, R. Cheng, X. Zhang, Y. Su, and Y. Jin. A Strengthened Dominance Relation Considering Convergence and Diversity for Evolutionary Many-Objective Optimization. *IEEE Transactions on Evolutionary Computation*, 23(2):331–345, 2019.
11. X. Zou, Y. Chen, M. Liu, and L. Kang. A New Evolutionary Algorithm for Solving Many-Objective Optimization Problems. *IEEE Transactions on Systems, Man, and Cybernetics, Part B*, 38(5):1402–1412, 2008.
12. J. Blank and K. Deb. Pymoo: Multi-Objective Optimization in Python. *IEEE Access*, 8:89497–89509, 2020.